

---

Lab 4 · Tema 4

## **Predicción de churn o abandono de clientes**

---

*Caso transversal: operador de telecomunicaciones*

Entorno: AWS Academy Learner Lab

Servicios: Amazon S3 · Amazon Athena · SageMaker Canvas / SageMaker Notebook

Duración estimada: 180 minutos · Nivel: intermedio

## 1. Idea general del laboratorio

Este laboratorio corresponde al Tema 4 y es la continuación natural del Lab 3. Allí formulamos el caso de predicción de churn, exploramos su viabilidad y construimos un prototipo rápido. Aquí entrenamos el modelo en profundidad: lo evaluamos con rigor, comparamos versiones, ajustamos el umbral de decisión, interpretamos qué variables pesan más y, finalmente, lo aplicamos a una lista de clientes activos para proponer una acción de retención concreta.

La predicción de abandono (churn) es uno de los casos de uso más rentables y mejor estudiados de la analítica en telecomunicaciones. Retener a un cliente suele ser mucho más barato que captar uno nuevo, de modo que anticipar quién está en riesgo permite actuar a tiempo. Pero también es un caso lleno de trampas: la fuga de información, el desequilibrio de clases, la confusión entre predicción y causa, y la tentación de automatizar decisiones que afectan a personas. Este laboratorio busca que el alumno construya el modelo y, al mismo tiempo, aprenda a desconfiar de él en la medida justa.

Ofrecemos dos caminos equivalentes para entrenar el modelo. El Camino A utiliza Amazon SageMaker Canvas, sin escribir código, ideal para comprender el ciclo y para perfiles no técnicos. El Camino B utiliza un Notebook de SageMaker con Python y scikit-learn, que da control total sobre el proceso y permite experimentos que Canvas no facilita. El alumno puede recorrer uno de los dos o ambos. Las partes de evaluación, experimentación y aplicación posteriores son comunes.

### Objetivos didácticos

Al finalizar el lab, el alumno debe ser capaz de:

1. Separar correctamente datos de entrenamiento, validación y aplicación, y explicar por qué es necesario.
2. Entrenar un modelo de clasificación de churn por al menos uno de los dos caminos (Canvas o Notebook).
3. Interpretar las métricas de un clasificador: accuracy, precision, recall, F1, matriz de confusión y AUC.
4. Comprender y detectar la fuga de información (data leakage) mediante un experimento controlado.
5. Ajustar el umbral de decisión en función del coste relativo de falsos positivos y falsos negativos.
6. Aplicar el modelo a clientes nuevos y traducir las predicciones en una acción de negocio responsable.
7. Distinguir las variables predictivas de las variables sobre las que tiene sentido intervenir.

## 2. Contexto, servicios y datasets

### Situación de partida

La operadora ha decidido lanzar una campaña de retención dirigida. En lugar de contactar a toda la base de clientes (caro e ineficaz), quiere concentrar el esfuerzo en quienes tienen mayor riesgo real de abandono. El equipo de datos dispone del histórico de clientes ya depurado (heredado de los Labs 2 y 3) y debe entregar dos cosas: un modelo entrenado y evaluado, y una lista priorizada de clientes activos sobre los que actuar.

### Servicios AWS utilizados

| Servicio                      | Función en el laboratorio                               |
|-------------------------------|---------------------------------------------------------|
| Amazon S3                     | Almacenar el dataset de entrenamiento y el de scoring.  |
| Amazon Athena                 | Comprobar el equilibrio de clases y preparar los datos. |
| SageMaker Canvas (Camino A)   | Entrenar y evaluar el modelo sin código.                |
| SageMaker Notebook (Camino B) | Entrenar y evaluar el modelo con Python y scikit-learn. |

Tabla 1. Servicios AWS empleados en el Lab 4.

### Datasets de partida

Se proporcionan dos ficheros, coherentes con el resto de la serie (mismos clientes, regiones y contratos):

| Fichero               | Filas | Contenido                                                                 |
|-----------------------|-------|---------------------------------------------------------------------------|
| telco_churn_train.csv | 900   | Clientes históricos CON la etiqueta churn (0/1). Para entrenar y validar. |
| telco_churn_score.csv | 150   | Clientes activos SIN etiqueta. Para predecir y proponer acción.           |

Tabla 2. Datasets del laboratorio.

La separación entre un fichero etiquetado (para aprender) y otro sin etiquetar (para aplicar) reproduce la realidad: entrenamos con el pasado conocido y aplicamos el modelo al presente, cuyo desenlace aún no conocemos. Esta distinción es el corazón del aprendizaje supervisado.

**Nota.** El fichero de scoring no tiene columna churn a propósito: son clientes que siguen activos y de los que todavía no sabemos si se irán. El objetivo del modelo es estimar esa probabilidad para poder actuar antes de que ocurra.

### 3. Preparación y carga

#### Paso 3.1. Iniciar AWS Academy y crear el bucket

1. Abre el Learner Lab, pulsa Start Lab y espera al círculo verde; abre la consola con AWS y confirma us-east-1.
2. Crea un bucket (por ejemplo unir-lab4-telco-churn-<iniciales>-<año>) con las carpetas train/, score/, athena-results/ y predictions/.
3. Sube telco\_churn\_train.csv a train/ y telco\_churn\_score.csv a score/.

#### Paso 3.2. Tabla de entrenamiento en Athena

Crea la base de datos y registra la tabla de entrenamiento para inspeccionarla:

```
CREATE DATABASE IF NOT EXISTS unir_telco_lab4;
CREATE EXTERNAL TABLE IF NOT EXISTS churn_train (
  customer_id          string,
  region               string,
  contract_type        string,
  monthly_price        double,
  customer_tenure_months int,
  avg_download_speed   double,
  expected_download_speed double,
  service_interruptions_30d int,
  interruption_minutes_30d double,
  num_tickets_30d      int,
  num_complaints_30d   int,
  call_center_contacts_30d int,
  avg_resolution_hours double,
  satisfaction_score    double,
  churn                int
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES ('separatorChar'=',', 'quoteChar'='"', 'escapeChar'='\\')
STORED AS TEXTFILE
LOCATION 's3://<tu-bucket>/train/'
TBLPROPERTIES ('skip.header.line.count'='1');
```

## Parte A. Preparar el problema de aprendizaje

### Paso A.1. Equilibrio de clases

Antes de entrenar, comprobamos cuántos clientes abandonan en el histórico. Un clasificador necesita suficientes ejemplos de la clase minoritaria:

```
SELECT churn, COUNT(*) AS clientes,  
       ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 1) AS pct  
FROM churn_train  
GROUP BY churn;
```

Con una tasa de churn en torno al 35-40 %, el problema está razonablemente equilibrado. Si la tasa fuera del 2-3 %, tendríamos un problema de clases muy desbalanceadas que exigiría técnicas específicas (remuestreo, ponderación de clases) y métricas adecuadas.

### Paso A.2. Entrenamiento, validación y prueba

Nunca se evalúa un modelo con los mismos datos con los que se entrenó: eso mide memoria, no capacidad de generalizar. Por eso dividimos el histórico en una parte de entrenamiento (con la que el modelo aprende) y otra de prueba (con la que lo evaluamos, simulando datos no vistos). Canvas hace esta división automáticamente; en el Notebook la haremos explícitamente con `train_test_split`.

| Conjunto      | Para qué sirve                                  | Quién lo ve                       |
|---------------|-------------------------------------------------|-----------------------------------|
| Entrenamiento | El modelo ajusta sus parámetros                 | El modelo, durante el aprendizaje |
| Validación    | Ajustar hiperparámetros y comparar modelos      | El proceso de entrenamiento       |
| Prueba (test) | Estimar el rendimiento real con datos no vistos | Solo al final, una vez            |
| Scoring       | Aplicar el modelo a clientes activos reales     | En producción, sin etiqueta       |

Tabla 3. Conjuntos de datos en el aprendizaje supervisado.

**Atención.** La regla de oro: el conjunto de prueba se mira una sola vez, al final. Si ajustas el modelo mirando repetidamente el test, acabas sobreajustando a él y la estimación de rendimiento deja de ser fiable.

## Parte B. Camino A — SageMaker Canvas (sin código)

**Nota.** Recorre este camino si prefieres una interfaz visual o si tu perfil no es de programación. Si vas a usar el Camino B (Notebook), puedes saltar a la Parte C. La disponibilidad de Canvas depende de la configuración de AWS Academy.

### Paso B.1. Importar el dataset

1. Abre SageMaker AI y entra en Canvas (puede estar dentro de SageMaker Studio).
2. En Datasets, pulsa Import data, elige Amazon S3 y selecciona telco\_churn\_train.csv en la carpeta train/.
3. Confirma la importación y revisa la previsualización.

### Paso B.2. Crear el modelo

4. Pulsa Create model, nómbralo telco\_churn\_lab4 y elige el tipo Predictive analysis.
5. Selecciona como Target column la columna churn. Canvas reconocerá una clasificación binaria (2 categorías).
6. En la previsualización, excluye customer\_id (es un identificador, no aporta patrón).

### Paso B.3. Entrenar con Standard build

A diferencia del Lab 3, donde usamos Quick build para un prototipo, aquí elegimos Standard build, que prueba varios algoritmos y entrega un modelo más sólido (tarda más, del orden de una a dos horas; planifica la sesión en consecuencia).

**Buena práctica.** Si el tiempo o el crédito de AWS Academy son limitados, Quick build sigue siendo válido a efectos docentes. Indica en el entregable qué modo usaste.

### Paso B.4. Leer los resultados

Cuando termine, Canvas muestra un cuadro con accuracy y, en la pestaña de detalle, precision, recall, F1, la matriz de confusión y la importancia de cada variable (column impact). Anota estos valores: los necesitarás en la Parte D y en el entregable.

### Paso B.5. Predecir sobre clientes nuevos

7. Importa también telco\_churn\_score.csv como un nuevo dataset.
8. En el modelo entrenado, usa Predict y selecciona el dataset de scoring (Batch prediction).
9. Canvas añade dos columnas: la predicción (0/1) y la probabilidad. Descarga el resultado: lo usaremos en la Parte F.

## Parte C. Camino B — SageMaker Notebook (Python)

**Nota.** Recorre este camino si quieres control total del proceso y entender qué ocurre por dentro. Da más libertad para el experimento de fuga de información de la Parte E.

### Paso C.1. Abrir un entorno de Notebook

1. En SageMaker AI, abre Studio (o Notebook instances, según el entorno de AWS Academy) y crea un notebook con kernel de Python 3 / Data Science.
2. Sube telco\_churn\_train.csv y telco\_churn\_score.csv al entorno, o léelos directamente desde S3.

### Paso C.2. Cargar los datos

```
import pandas as pd

# Opción 1: leer directamente desde S3
bucket = 's3://<tu-bucket>'
train = pd.read_csv(f'{bucket}/train/telco_churn_train.csv')
score = pd.read_csv(f'{bucket}/score/telco_churn_score.csv')

print(train.shape, score.shape)
print(train['churn'].value_counts(normalize=True).round(3))
```

### Paso C.3. Preparar variables (features)

Separamos la variable objetivo, derivamos la brecha de velocidad y codificamos las variables categóricas con one-hot. Quitamos customer\_id porque es un identificador.

```
import numpy as np

train['speed_gap'] = train['expected_download_speed'] - train['avg_download_speed']
score['speed_gap'] = score['expected_download_speed'] - score['avg_download_speed']

y = train['churn']
X = train.drop(columns=['customer_id', 'churn'])

# one-hot de categóricas
X = pd.get_dummies(X, columns=['region', 'contract_type'], drop_first=True)
print(X.shape)
```

### Paso C.4. Dividir en entrenamiento y prueba

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
print(X_train.shape, X_test.shape)
```

**Buena práctica.** stratify=y mantiene la misma proporción de churn en entrenamiento y prueba. Es importante cuando hay desequilibrio de clases.

### Paso C.5. Entrenar dos modelos

Entrenamos una regresión logística (lineal, muy interpretable) y un random forest (no lineal, más potente), y los compararemos.

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# Regresión logística (con escalado)
logit = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000, class_weight='balanced'))
logit.fit(X_train, y_train)

# Random forest
rf = RandomForestClassifier(
    n_estimators=300, max_depth=8,
    class_weight='balanced', random_state=42)
rf.fit(X_train, y_train)
```

## Paso C.6. Evaluar

```
from sklearn.metrics import (classification_report,
                             confusion_matrix, roc_auc_score)

for nombre, modelo in [("Logística", logit), ("Random Forest", rf)]:
    pred = modelo.predict(X_test)
    proba = modelo.predict_proba(X_test)[:, 1]
    print(f"==== {nombre} =====")
    print(confusion_matrix(y_test, pred))
    print(classification_report(y_test, pred, digits=3))
    print("AUC:", round(roc_auc_score(y_test, proba), 3))
    print()
```

## Paso C.7. Importancia de variables

```
import pandas as pd

importancias = pd.Series(rf.feature_importances_, index=X.columns)
print(importancias.sort_values(ascending=False).head(10))
```

Preguntas: ¿qué variables encabezan la importancia? ¿Coinciden con lo que esperabas tras la exploración del Lab 3? ¿Aparece satisfaction\_score como dominante? Si es así, lee con atención la Parte E.

## Parte D. Evaluación profunda y umbral de decisión

### Paso D.1. Entender la matriz de confusión

La matriz de confusión cruza lo que el modelo predijo con lo que realmente ocurrió. En churn, sus cuatro celdas tienen consecuencias de negocio muy distintas:

|                    | Predijo: se queda                | Predijo: se va                         |
|--------------------|----------------------------------|----------------------------------------|
| Realidad: se queda | Verdadero negativo (correcto)    | Falso positivo (descuento regalado)    |
| Realidad: se va    | Falso negativo (cliente perdido) | Verdadero positivo (retención posible) |

Tabla 4. Matriz de confusión interpretada para el churn.

### Paso D.2. Precision y recall: el dilema

La precision responde: de los clientes que el modelo marcó como "en riesgo", ¿cuántos lo estaban realmente? El recall responde: de todos los que realmente se fueron, ¿a cuántos detectó el modelo? Subir uno suele bajar el otro. En retención, normalmente preferimos recall alto (no dejar escapar a quien se va), aceptando algunos falsos positivos (ofrecer retención a quien no la necesitaba).

### Paso D.3. Ajustar el umbral

Por defecto, un clasificador marca "se va" cuando la probabilidad supera 0,5. Pero ese umbral es una decisión de negocio, no una ley. Si un falso negativo (perder un cliente) es mucho más caro que un falso positivo (un descuento innecesario), conviene bajar el umbral para capturar más casos en riesgo. En el Notebook:

```
import numpy as np

proba = rf.predict_proba(X_test)[: , 1]
for umbral in [0.3, 0.4, 0.5, 0.6]:
    pred_u = (proba >= umbral).astype(int)
    tp = int(((pred_u == 1) & (y_test == 1)).sum())
    fp = int(((pred_u == 1) & (y_test == 0)).sum())
    fn = int(((pred_u == 0) & (y_test == 1)).sum())
    prec = tp / (tp + fp) if (tp + fp) else 0
    rec = tp / (tp + fn) if (tp + fn) else 0
    print(f"umbral {umbral}: precision {prec:.2f}, recall {rec:.2f}, FP {fp}, FN {fn}")
```

Preguntas: ¿qué umbral elegirías si el coste de perder un cliente quintuplica el de un descuento? ¿Y si la campaña de retención fuera muy cara? El umbral óptimo depende de la economía del problema, no del modelo.

**Nota.** En Canvas, el ajuste de umbral es más limitado, pero puedes simular el efecto ordenando las predicciones por probabilidad y eligiendo cuántos clientes del top contactar. El concepto es el mismo.

## Parte E. Experimento de fuga de información

La fuga de información (data leakage) ocurre cuando el modelo usa una variable que, en la práctica, no estaría disponible en el momento de predecir, o que está tan ligada al resultado que "hace trampa". En nuestro dataset, `satisfaction_score` es sospechosa: la satisfacción se mide a menudo cuando el cliente ya está decidido, de modo que predecir el abandono con ella es casi tautológico.

### Paso E.1. Entrenar sin `satisfaction_score` (Notebook)

```
X_sin = X.drop(columns=['satisfaction_score'])
Xtr, Xte, ytr, yte = train_test_split(
    X_sin, y, test_size=0.25, random_state=42, stratify=y)

rf_sin = RandomForestClassifier(
    n_estimators=300, max_depth=8,
    class_weight='balanced', random_state=42).fit(Xtr, ytr)

from sklearn.metrics import roc_auc_score
proba_sin = rf_sin.predict_proba(Xte)[: , 1]
print("AUC sin satisfaction:", round(roc_auc_score(yte, proba_sin), 3))
```

### Paso E.2. Comparar

Compara el AUC del modelo con `satisfaction_score` (Paso C.6) y sin ella. Si el rendimiento cae mucho al quitarla, el modelo dependía fuertemente de una variable problemática. Si apenas cambia, el resto de variables (interrupciones, reclamaciones, antigüedad) ya capturan la señal de forma más accionable.

| Versión del modelo                  | ¿Qué mide?                                  | Utilidad real                                     |
|-------------------------------------|---------------------------------------------|---------------------------------------------------|
| Con <code>satisfaction_score</code> | Rendimiento optimista, posiblemente inflado | Engañosa si la satisfacción llega tarde           |
| Sin <code>satisfaction_score</code> | Rendimiento más honesto y accionable        | Permite intervenir antes, sobre causas operativas |

Tabla 5. Efecto de la fuga de información.

**Atención.** Una variable muy predictiva no siempre es una buena variable. Si solo está disponible cuando ya es tarde para actuar, o si es casi un sinónimo del resultado, su poder predictivo es ilusorio a efectos de gestión.

### Paso E.3. Variables predictivas vs variables accionables

Termina esta parte con una reflexión clave: algunas variables predicen bien pero no son accionables (no podemos cambiar la antigüedad de un cliente), mientras que otras predicen y además señalan dónde intervenir (las interrupciones de servicio o los tiempos de resolución sí se pueden mejorar). Un buen modelo de churn no solo ordena clientes por riesgo: ayuda a entender sobre qué palancas operativas merece la pena actuar.

## Parte F. Aplicar el modelo y proponer acción

Un modelo solo aporta valor cuando se traduce en decisiones. Aplicamos el modelo a los 150 clientes activos del fichero de scoring y construimos una lista priorizada de retención.

### Paso F.1. Puntuar a los clientes activos (Notebook)

```
X_score = pd.get_dummies(
    score.drop(columns=['customer_id']),
    columns=['region', 'contract_type'], drop_first=True)
# alinear columnas con las del entrenamiento
X_score = X_score.reindex(columns=X.columns, fill_value=0)

score['prob_churn'] = rf.predict_proba(X_score)[ :, 1]
ranking = score.sort_values('prob_churn', ascending=False)
ranking[['customer_id', 'region', 'contract_type',
        'service_interruptions_30d', 'num_complaints_30d',
        'prob_churn']].head(20)
```

### Paso F.2. Guardar la lista en S3

```
ranking.to_csv('churn_predictions.csv', index=False)
# subir a la zona predictions/ del bucket
import boto3
boto3.client('s3').upload_file(
    'churn_predictions.csv', '<tu-bucket>',
    'predictions/churn_predictions.csv')
```

Si usaste el Camino A (Canvas), ya tienes esta lista del Paso B.5; ordénala por probabilidad descendente.

### Paso F.3. Del ranking a la acción

La lista priorizada no es el final, sino el principio de una decisión humana. Para los clientes del tramo de mayor riesgo, el equipo debe decidir qué acción es adecuada, y esa acción debería atender a la causa probable, no solo al riesgo. Un cliente con muchas interrupciones necesita una solución técnica, no un descuento; un cliente con queja de facturación necesita una revisión administrativa.

| Perfil del cliente en riesgo          | Acción recomendada                   | Quién decide        |
|---------------------------------------|--------------------------------------|---------------------|
| Muchas interrupciones, baja velocidad | Revisión técnica de la línea/zona    | Operaciones de red  |
| Resolución lenta, varios contactos    | Gestor de incidencia dedicado        | Soporte             |
| Queja de facturación                  | Revisión administrativa de la cuenta | Atención al cliente |
| Sin causa técnica clara, precio alto  | Oferta de retención (con criterio)   | Comercial           |

Tabla 6. De la predicción a la acción según la causa probable.

**Atención.** Ofrecer descuentos automáticos a todos los clientes en riesgo es la respuesta perezosa y, a menudo, contraproducente: premia a quien amenaza con irse e ignora el problema real. La predicción debe abrir una conversación, no cerrarla.

## Parte G. Entregable del alumno

El alumno entregará un documento de entre cinco y siete páginas con:

1. Indicación del camino seguido (Canvas, Notebook o ambos) y del modo de entrenamiento.
2. Equilibrio de clases y estrategia de división de datos empleada.
3. Métricas del modelo en el conjunto de prueba: accuracy, precision, recall, F1, AUC y matriz de confusión, con interpretación.
4. Tabla de importancia de variables y comentario sobre cuáles son predictivas y cuáles accionables.
5. Resultado del experimento de fuga de información (AUC con y sin satisfaction\_score) y conclusión.
6. Análisis del umbral de decisión: qué umbral elegirías y por qué, en función del coste de los errores.
7. Lista priorizada de los 20 clientes de mayor riesgo del fichero de scoring, con la acción propuesta para los cinco primeros según su causa probable.
8. Reflexión final sobre los riesgos de automatizar la decisión de retención.

### Criterios de evaluación

| Criterio                   | Peso | Insuficiente            | Aceptable                        | Excelente                               |
|----------------------------|------|-------------------------|----------------------------------|-----------------------------------------|
| Entrenamiento correcto     | 20%  | Sin división train/test | Modelo entrenado y evaluado bien | Compara modelos o modos con rigor       |
| Interpretación de métricas | 20%  | Solo reporta accuracy   | Interpreta precision/recall      | Relaciona métricas con coste de negocio |
| Fuga de información        | 20%  | No la aborda            | Hace el experimento              | Concluye sobre accionabilidad           |
| Umbral de decisión         | 15%  | Usa 0,5 sin reflexión   | Justifica un umbral              | Razona sobre la economía de los errores |
| Scoring y acción           | 15%  | Solo predice            | Lista priorizada                 | Acción ligada a la causa probable       |
| Reflexión y formato        | 10%  | Pobre                   | Adecuada                         | Crítica, profesional, bien presentada   |

Tabla 7. Criterios y rúbrica de evaluación.

## Parte H. Limpieza de recursos y cierre

### Paso H.1. Detener recursos de cómputo

- Si usaste un Notebook o Studio, detén la instancia o el kernel (Stop) para que no siga consumiendo crédito; este punto es crítico en AWS Academy.
- Si usaste Canvas, haz Log out de Canvas para liberar la sesión.

### Paso H.2. Eliminar objetos de Athena

```
DROP TABLE IF EXISTS churn_train;  
DROP DATABASE IF EXISTS unir_telco_lab4;
```

### Paso H.3. Vaciar S3 y End Lab

1. Vacía y elimina el bucket si no vas a reutilizarlo (conserva predictions/ si quieres guardar la lista).
2. Pulsa End Lab y verifica que el círculo pasa a rojo.

**Atención.** Las instancias de Notebook y las sesiones de Canvas son, con diferencia, lo que más crédito consume en AWS Academy. Detenerlas explícitamente antes de End Lab es la diferencia entre conservar crédito para los siguientes labs o quedarte sin él.

## Cierre didáctico

Has entrenado, evaluado y aplicado un modelo de predicción de abandono, y lo has hecho con una mirada crítica que es tan importante como la técnica. Has visto que un buen modelo no es el que tiene mayor accuracy, sino el que ayuda a tomar mejores decisiones: por eso has interpretado la matriz de confusión en términos de coste, has ajustado el umbral según la economía del problema, has desenmascarado una fuga de información y has distinguido las variables que predicen de las que permiten intervenir.

Sobre todo, has aprendido que el modelo entrega una lista de clientes en riesgo, pero no una orden: la decisión de qué hacer con cada cliente sigue siendo humana y debe atender a la causa, no solo al síntoma. En los próximos temas dejaremos la analítica de clientes para mirar la estructura del sistema como una red (Labs 5, 6 y 7), analizar el texto de las reclamaciones con modelos de lenguaje (Lab 8) y, finalmente, abordar la pregunta que este modelo no puede responder por sí solo: qué causa realmente el abandono (Lab 9).

## Referencias y lecturas complementarias

- Amazon Web Services. (2024). Amazon SageMaker Canvas User Guide. <https://docs.aws.amazon.com/sagemaker/latest/dg/canvas.html>
- Amazon Web Services. (2024). Amazon SageMaker — Train a model. <https://docs.aws.amazon.com/sagemaker/>
- Géron, A. (2022). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly Media.

- Kaufman, S., Rosset, S., & Perlich, C. (2012). Leakage in data mining. ACM TKDD, 6(4).
- Provost, F., & Fawcett, T. (2013). Data science for business. O'Reilly Media.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. JMLR, 12, 2825-2830.